

3D Reconstruction from Video: A Comparison Between NeRFs, Photogrammetry, and Gaussian Splatting

Ian Christensen, Damian Czernikowski, Arjun Shanmugas

Department of Electrical and Computer Engineering

Robotics & Computer Vision

14:332:472:01

Rutgers University

Abstract—This paper compares three popular techniques for 3D reconstruction from video used in our project: Neural Radiance Fields (NeRFs), photogrammetry, and Gaussian splatting. In the project we explore their differences in reconstructing smaller objects. We also focus on proper data collection methods for good reconstruction accuracy. Through experimentation with various datasets, including objects like bananas and mini-figures, we examine the strengths and limitations of each method. Key findings highlight the importance of camera motion, lighting conditions, and dataset selection in achieving high-quality reconstructions.

Index Terms—NeRFs, Photogrammetry, Gaussian Splatting, 3D Reconstruction, Computer Vision

I. INTRODUCTION AND BACKGROUND

“Disclaimer: A 15-minute video was produced for this project, the data collected will be better represented in the video than in the figures in this paper.”

In our project we were given 5 options to choose to explore, or any combination of the options, options were as follows:

- RL (Reinforcement Learning)
- NeRF with colmap
- Other 3D Reconstruction
- Transfer Learning (to fine-tune on existing DL models)
- Foundation Model (to use & evaluate)

For our project we chose to go with a 3D reconstruction using NeRF with colmap, and other methods, because we thought it would be fun. For the work we ended up doing, we should have begun much sooner. Our professor did warn us however. We also shot ourselves in the foot a bit and wasted a couple of days by being a bit too over-eager in our initial plan, and perhaps not realizing that soon enough. Regardless, we all thoroughly enjoyed this project and the things we learned along the way.

Initially we set the scope of the project with the thought we would be able to fully 3D reconstruct the exterior and interior of the EE building on Busch campus. We learned over the course of about 2-4 days that this was not going to be possible. As such, we set our sights on creating a 3D reconstruction of smaller objects, and comparing different reconstruction methods on our image datasets. Later in

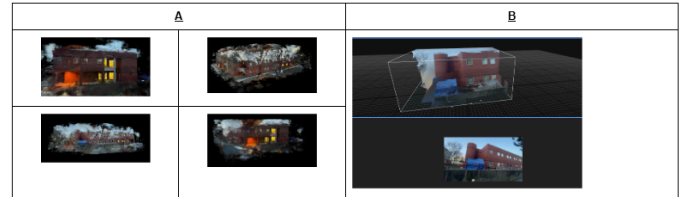


Fig. 1. Initial reconstructions from video with mediocre image quality and improper data collection methods A: NeRF reconstruction from the corresponding imageset B: Photogrammetry reconstruction from corresponding imageset, note: lower half is reference real/raw image

discussion, we will touch on how we could potentially achieve our initial plan. It is also important to note that all 3D reconstructions done in this project were done on our personal machines using our personal hardware (i.e. GPU's, CPU's, ect.)

II. INITIAL RESEARCH & METHODOLOGY

When we began collecting preliminary data for the EE building we didn't spend long enough researching proper data collection methods. We took a few images of the building and went back to process them only to be met with errors. After further research we realized that to be able to fully assess even just the exterior of the EE building, we would most likely need a drone, and permission from the university & potentially Rutgers police as well. Partially due to the landscape surrounding the EE building in particular, but also due to other reasons revealed in the following section. Realizing we would be unable to reconstruct the EE building as we had initially hoped, we set our sights on reconstructing smaller objects instead. This yielded us much better results.

III. DATA COLLECTION

A. Initial Mistakes & Learnings

We initially thought a handful of pictures from different angles of the ECE building would be enough to do a rough 3D reconstruction and go from there. Unfortunately, we were not even able to run this dataset through colmap. We learned

that the best way for us to go about this would be to record a video and use ffmpeg to parse the video into frames. It is important to consider ffmpeg does not have any consideration in the frames it chooses, and some may be less than ideal. It is recommended to view the imageset and remove blurry frames before processing with colmap. We did this with "FastStone Image Viewer". We aimed for around 100-300 frames for any given object. More frames typically result in higher quality reconstructions, but not always. We also learned that there are effective and ineffective ways to capture images, particularly regarding camera angles and movement around the object during filming. Initially we made a lot of angular panning movements for some of our first reconstructions and learned that these types of motions can be harmful to the quality of the end result. The most effective way to capture an object is to image the same features from multiple angles, but with a relatively similar distance from the object. In the figure below (fig. 2) we show some good and bad camera motions for capturing an object for 3D reconstruction.

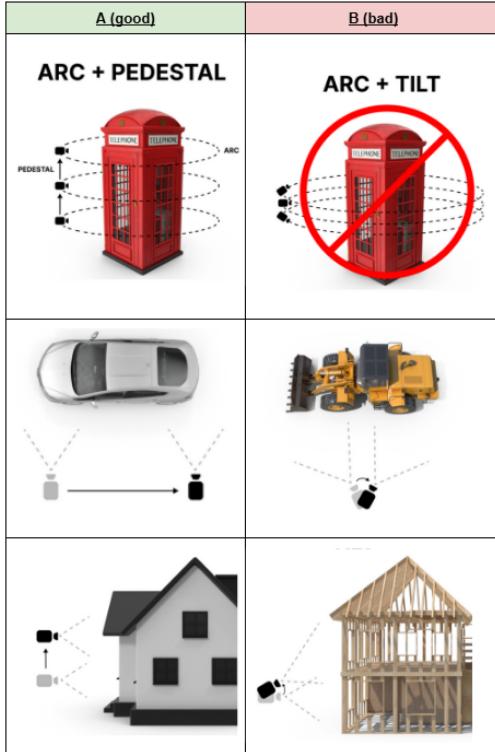


Fig. 2. Good and bad camera motions for capturing an object in 3D reconstruction. A: Good motions include panning around the object horizontally without angling the camera vertically & panning across the object. B: Bad camera movements include pivoting the camera in one place.

Lighting is also a relatively important factor, as it accounts for the features found in the images. Strong background light with minimal shadows seemed to provide the best results. It is also a reason why we found you must rotate the camera around the object, rather than rotate the object and leave the camera stationary. The way the light falls upon

the object and therefore reflects back defines its feature. If the object is rotated, the light falling upon the object is constantly changing, and as a result, colmap may struggle to create feature correspondences, or even be unable to match keypoints. We tested this, and although colmap was able to execute and produce a transforms.json file, the rendered reconstruction was hardly recognizable (using a number of high quality images).

B. Our Methodology

To reconstruct a good model, we imagined a sphere around our object, with our object in the center. Because the object was sitting on a platform or surface such as the ground, we can typically neglect the bottom half of the sphere that our object is "inside", leaving us with a hemisphere (fig. 3).

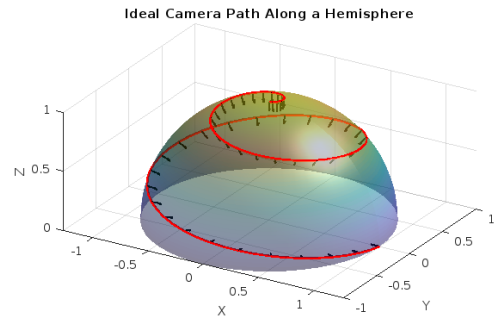


Fig. 3. Our methodology for capturing good datasets for 3D reconstruction of an object using video. By rotating around the object and spiraling down while maintaining distance and consistent focal point, we found our results to be very good. (More applicable to smaller objects.)

To achieve a good reconstruction, we want to move our camera around the surface of this hemisphere, while keeping the camera pointed at our object, or the center of the hemisphere. We found that two rotations around the object, in a spiral motion starting from near the top, but not quite directly above, and moving down to near the bottom gave good results. We found it relatively difficult to achieve this with objects on the ground, or even on a table/counter-top. In the end, we found the easiest way to image smaller objects was to pedestal them up to about chest height, leaving plenty of room to physically rotate around the object. This can be achieved in many ways. Two convenient ways used in this project were a speaker stand, and a short plant stand stacked on other objects.

C. Chosen Datasets

We chose to image many objects over the course of this project. However, we learned that there are certain objects that typically yield better results. Objects with minimal or no reflective material/surfaces tend to be desirable for quality reconstruction results. As well, objects lacking any translucent surfaces are typically a wise choice. As such, as the project progressed, our choice of objects adapted as well. In this

paper we chose to represent 4 of our best datasets (fig. 4) for comparison between different 3D reconstruction methods:

- 1) **Bananas:** The bananas used were a bunch of 3, quite ripe bananas, with a number of brown patches and surface features. They were placed on a speaker stand.
- 2) **Silent Minifigure:** A minifigure of a character named Silent from the video game “Slay the Spire” made from Youtooz. Placed on hardwood floor.
- 3) **Peace Pillow & Pinecone:** A very small “throw” pillow with the word Peace written on it, as well as a pinecone laid overtop. Placed on a speaker stand.
- 4) **Cannabis Flower:** The flower from a cannabis plant. Many concave and convex surface features, with very fine detail, as well as colorful leaves. Placed on a small plant stand on top of a printed checkerboard pattern.

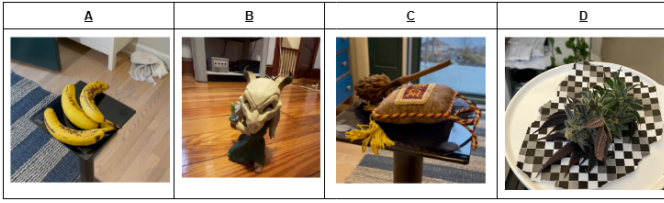


Fig. 4. Raw/Real images of the objects from chosen datasets. A: A bunch of bananas. B: Silent, a video-game character. C: A pillow and a pinecone. D: The flower from a cannabis plant, including leaves

Below in (fig. 5) is our camera positions for each image in each dataset. This was calculated with some simple lines of python using the “transforms.json” file output from colmap, which we will discuss shortly. You can somewhat see the “spiral” motion we aimed for when capturing video of each object.

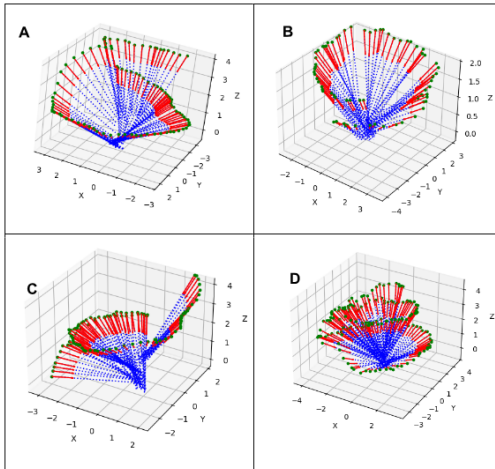


Fig. 5. Camera positions for each image in the corresponding dataset A: Bananas. B: Silent. C: Peace. D: Flower. Green dots are camera position, red vectors are camera orientation, and dashed blue line is a continuation of the cameras view angle.

IV. NERF RECONSTRUCTIONS

A. Background

Neural Radiance Fields (NeRF) is a deep learning model that reconstructs 3D spaces from a set of 2D images. It does this by mapping 3D coordinates (x,y,z) and the camera’s view direction to output the color (r,g,b) and the volume density of each point in the 3D space. The volume density determines how solid or opaque a point appears. NeRF learns this representation using a neural network, typically a multi-layer perceptron (MLP). To create the final 3D rendering, NeRF employs volume rendering. This technique shoots rays through the scene, sampling multiple points along each ray. The model predicts the color and density for each sampled point, and these predictions are combined to calculate the final rendered color of the ray, resulting in a realistic 3D representation.

colmap: is an open source software that is critical step in 3D reconstruction for NeRF’s as well as other applications. It detects distinct features in the images given to it, and matches them across the different images. It also estimates the camera poses and generates a point cloud. To achieve this, colmap goes through a number of steps, which can be quite time consuming on some machines as we learned in this project.

- 1) **Feature Detection & Camera Calibration:** Utilizing feature detectors like SIFT (scale-invariant feature transform), colmap identifies distinct repeatable keypoints in each image that can later be matched between images. Following steps rely on this steps execution. This step also calculates the intrinsic camera parameters such as focal length.
- 2) **Feature Matching:** Colmap then finds correspondences between the features detected by SIFT in pairs of images and matches them. In our case, matching was sometimes exhaustive, meaning every image was compared to one another, which can be quite time consuming. Data preparation can be a useful tool to work around this.
- 3) **Initialization:** Colmap selects an initial pair based on those with the highest number of matched features. It then starts estimating the relative camera pose (rotation & translation) between the two images using the matched features and typically an algorithm like RANSAC.
- 4) **Incremental Reconstruction:** Colmap continues along the previous step, iterating through all the images. For each new image, colmap again estimates the pose relative to the existing “3D Structure”. New 3D points are triangulated and added to the sparse point cloud.
 - Sparse point cloud is the set of 3D points in a dataset which are relatively far apart from each other. For an ideal reconstruction, these points should be closer together to form a “dense point cloud”. The denser the points are, the more accurate of a reconstruction will be produced.

- 5) **Bundle Adjustment:** Lastly, colmap optimizes all the camera parameters & 3D points via bundle adjustment to minimize error. This is done with a non-linear least squares optimization over the entire dataset.

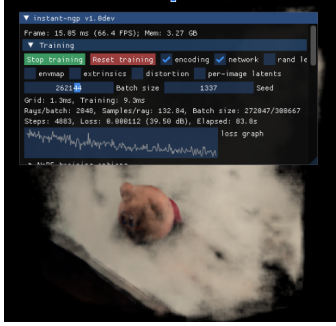


Fig. 6. Our first successful reconstruction using NeRFs, image is noisy and object is not distinct. This led us to further explore proper data collection methods. Object is a small Kirby figurine from the Kirby video-game series, imaged on a white counter-top.

Above in (fig. 6) we show our first successful reconstruction of an object using NeRFs, this was done immediately after failing to run our preliminary pictures of the EE building through colmap. This was honestly a very exciting moment for us all, but it also highlighted that we still had much to learn about proper data collection, as the image is noisy and the object is not very distinct. Once we learned proper techniques & camera movements however, we were able to produce much better results.

B. Results

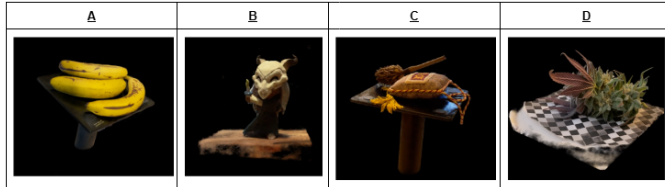


Fig. 7. 3D Reconstruction of chosen objects utilizing NeRFs, powered with NVidia's INGP. Video will show better detail. A: Bananas B: Silent C: Peace D: Flower

After figuring out how to properly collect data for NeRF reconstructions as well as others, we were very pleased with the results (fig. 7) we received. All datasets represented in this paper looked very good utilizing INGP to reconstruct. For training, we aimed for around 10k steps as we noticed the loss function typically plateauing around 6-8k steps.

Looking at images A through D in (fig. 7), NeRF was able to successfully recreate the objects with extreme detail. Being able to fully wrap around the object improves from what was previously seen in (fig. 1 & fig. 4). Before, there were a lot of gaps and uncertainties since NeRF did not have the entire object to map. However, by using the proper data collecting techniques, colmap had a much easier time identifying the

feature vectors in each frame and NeRF was able to train on a more detailed dataset.

V. STRUCTURE FROM MOTION (PHOTOGRAMMETRY) RECONSTRUCTIONS

A. Background

Photogrammetry, or structure from motion, is another method that can utilize the same datasets we have been producing to also create 3D reconstructions. It differs from NeRFs with respect to how it computes and represents the scene or object. Photogrammetry typically requires less computational power to execute than NeRFs. Unlike NeRFs, which creates a volumetric rendering of the scene, photogrammetry creates a point cloud, which can be converted into a mesh/model and textured utilizing the base images.

To do 3D reconstructions of our datasets utilizing structure from motion we made use of an application produced by EpicGames titled "RealityCapture". The application made the process quite simple and streamlined it for us. All we really had to do was drag and drop our image-set created with ffmpeg, the same image-set used with colmap for NeRFs, into the program, and then step through the simple functions.

B. Results

Our results (fig. 8) from photogrammetry came out pretty well, some better than others. We noticed that smoother objects tended to be better represented with this reconstruction method. Also interesting to note is photogrammetry seemed to struggle with reflective surfaces much more than NeRFs. In the example with the bananas, NeRFs did an excellent job capturing the reflections off the glass surface they were sat on, while photogrammetry struggled with these areas and left holes in the model in their place (fig. 6: A, C)

Our favorite result from our datasets utilizing photogrammetry was the Silent minifigure. It had the cleanest surface of all the reconstructions and the least errors. The bananas also reconstructed quite well. Either could be exported and printed as 3D models or used in a video game as an object. The peace pillow and flower did not reconstruct as well in photogrammetry. They had a number of surface issues, and in some locations major geometrical inaccuracies.

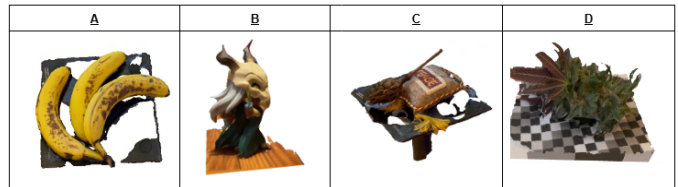


Fig. 8. 3D reconstruction of chosen objects utilizing photogrammetry(structure from motion), powered by EpicGames "RealityCapture". A: Bananas B: Silent C: Peace D: Flower. A & B reconstructed well. C & D contain number of surface & geometrical inaccuracies

Overall it was satisfying to see these models produced using a different method. However, we prefer the NeRF reconstructions as they seemed more consistent and the process for producing them as well as the program utilized gave us much more control over the entire reconstruction process from beginning to end result.

VI. GAUSSIAN SPLAT RECONSTRUCTIONS

A. Background

Gaussian Splatting is a technique used to render a 3D scene from 2D images. Using groups of 3D gaussian ellipsoids to represent a 3D space, providing a computationally efficient method for reconstruction. The setup for this method involve a multitude of 2D images run through a colmap to generate the location, orientation, and a sparse/dense 3D point cloud of the scene. This 3D point cloud scene is used as the initialization for a collection of 3D Gaussian ellipsoids. Each gaussian is defined by its position, orientation, anisotropy (shape along different axes), color, and density. Optimizing further refines the gaussian's parameter is used to ensure an accurate, "optimization of the properties of the 3D Gaussians... interleaved with adaptive density control steps, where we add and occasionally remove 3D Gaussians during optimization. The optimization procedure produces a reasonably compact, unstructured, and precise representation of the scene" [3]. Through optimization, gaussian splatting allows for real time rendering with superior visual quality.

B. Results

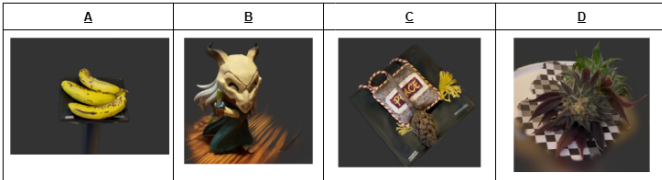


Fig. 9. 3D reconstruction of chosen objects utilizing gaussian splatting, powered by Jawset's "PostShot". A: Bananas B: Silent C: Peace D: Flower

We were very impressed with the results from gaussian splatting; for some of us, this was our favorite reconstruction method. Gaussian splat results tended to be a "smoother" version of the NeRF reconstructions, and this makes sense from what we know about how gaussian splatting works. The results from this method were perhaps some of the more photorealistic/accurate representations, at least from certain distances.

VII. DISCUSSION

A. Comparison Between Methods

NeRF, photogrammetry, and gaussian splatting are all widely used methods for 3D reconstruction but each have their own unique features. As said before, NeRF creates feature vectors and matches it with the other features in every other image, thus creating a splash of several colors that orient themselves to create the object given the perspective. NeRF

also allows for vague estimates of how the given object would look at certain perspectives not captured by the images. While NeRF's capabilities to estimate are what makes it one of the stronger reconstruction capabilities, its downsides come into play with its intensive computations. For large-scale models with a lot of images, it can take hours or even days to process. Photogrammetry, however, looks at the surfaces of objects and matches the features from sparse point clouds. Something that isn't shown in this paper is how the sky is never modelled in photogrammetry, but in Figure 1, you can see how NeRF is trying to match those colors together. This basically takes away some of the sizing issues when it comes to reconstructing larger objects and takes longer to process. If photogrammetry was used, it would've gone faster since it would have considered shorter feature vectors. As for gaussian splatting, this method is almost like if one were to put the good things from NeRFs and photogrammetry all together. Gaussian splatting has the capabilities to do accurate estimates given the areas that each cloudcovers, and has enough knowledge to create smooth and realistic textures given its ability to detect surfaces in its splats. Unfortunately, if using sparse point cloud data, then the splats may leave gaps and awkward textures in its model [6].

B. Initial Plan

Earlier in the paper we mentioned we would discuss potential ways to achieve our initial plan of reconstructing the entirety of the ECE building, exterior & interior. Given enough time, this research project could have been more detailed in the reconstruction of the Electrical Engineering Building. What was missing was the ability to use a drone to properly take videos of the perimeter of the roof of the building to fully encapsulate it. Additionally, with a drone, the stability and pathing of recording would have been far more accurate and thus given better generations since walking around the building had a multitude of obstacles. Additionally, to model the inside of the building would have required the merging of several reconstructions which weren't in the prospects of the given time. This would have required further computations as another NeRF would need to be trained in order to average out the confidence scores of the edges/colors.

VIII. CONCLUSIONS

This paper examines various 3D reconstruction AI models, including NeRF, photogrammetry, and gaussian splatting, by applying them to both complex and simple objects. Through these reconstructions, we compared their strengths, limitations, and the abnormalities in each method introduced. We also explored the importance of data collection techniques and their impact on the quality of the reconstructions. One of the primary challenges encountered was ensuring high-quality video data, as blurry frames were taken and thus resulted in unrecognizable reconstructions. However, it was intriguing to observe how these models could still achieve relatively accurate results even with suboptimal input. This highlights the robustness and adaptability of these reconstruction methods,

while also acknowledging the importance of precise data acquisition for achieving optimal outcomes.

REFERENCES

- [1] NVlabs, "GitHub - NVlabs/instant-ngp: Instant neural graphics primitives: lightning fast NeRF and more," GitHub. [Online]. Available: <https://github.com/NVlabs/instant-ngp>
- [2] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis," arXiv.org. [Online]. Available: <https://arxiv.org/abs/2003.08934>
- [3] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian Splatting for Real-Time Radiance Field Rendering," arXiv.org. [Online]. Available: <https://arxiv.org/abs/2308.04079>
- [4] "Reconstructing indoor spaces with NeRF." Accessed: Dec. 18, 2024. [Online]. Available: <https://research.google/blog/reconstructing-indoor-spaces-with-nerf/>
- [5] NVlabs, "GitHub - NVlabs/instant-ngp: Instant neural graphics primitives: lightning fast NeRF and more," GitHub. [Online]. Available: <https://github.com/NVlabs/instant-ngp>
- [6] "Sparse vs. Dense Data – the Power of Points and Clouds – Revware Portable CMMs." Accessed: Dec. 18, 2024. [Online]. Available: <https://revware.net/sparse-vs-dense-data-the-power-of-points-and-clouds/>
- [7] W. Zhang, Z. Li, Y. Guo and F. Wang, "Combine Neural Radiance Fields in Different Reference Coordinates," 2023 China Automation Congress (CAC), Chongqing, China, 2023, pp. 9067-9072, doi: 10.1109/CAC59555.2023.10450329.